

1. Pseudocode Algorithm A

Pseudocode Algorithm A: FIFO Queue

Algorithm A: FIFO Queue

Input: Order ใหม่

Output: Order ที่ถูกนำไปเตรียมอาหาร

1. Create FIFO Queue
2. รับ order ใหม่
3. เพิ่ม order เข้า queue
4. ตรวจสอบว่า queue ไม่ว่าง
5. ถ้า queue ไม่ว่าง
 - Return false
6. ถ้า queue ไม่ว่าง
 - นำ order ตัวหน้าสุดออกด้วย poll()
 - ประมวลผล order
 - บันทึก Waiting Time
7. ตรวจสอบว่า order ต่อไปอยู่ใน queue
8. คืนค่า order
9. End

หลักการ

- โครงสร้างไม่ซับซ้อน
- ใช้หลัก First In, First Out
- Order ที่เข้ามาก่อนจะถูกจัดการก่อน
- รองรับการเพิ่มและนำ Order ออกจาก Queue
- มีความยุติธรรมในการจัดลำดับ

ข้อจำกัด

- Express Order ไม่สามารถแซง Order ที่มาก่อนได้
- ถ้ามี Normal หรือ Large เข้ามาก่อน Express ต้องรอตตามลำดับ
- ไม่เหมาะกับกรณีที่ต้องการให้ Order เร่งด่วนพาทาก่อน
- ต้องจัดการ Queue ให้ถูกต้อง

ลำดับตัวอย่าง Algorithm A:

01 → 02 → 03 → 04 → 05

2. Pseudocode Algorithm B

Pseudocode Algorithm B: Priority Queue

Algorithm B: Priority Queue

Input: Order ใหม่

Output: Order ที่ถูกเลือกตาม Priority

1. Create PriorityQueue
2. รับ Order ใหม่
3. กำหนด Priority ให้กับ Order
4. เพิ่ม Order เข้า PriorityQueue
5. ตรวจสอบ Priority ของ Order ใน Queue
6. ถ้า Queue ว่าง
 - Return false
7. ถ้า Queue ไม่ว่าง
 - เลือก Order ที่มี Priority สูงสุด
 - ถ้า Priority เท่ากัน ให้ดู Order Time
 - นำ Order ออกจาก Queue ด้วย poll()
 - ประมวลผล Order
 - บันทึก Waiting Time
8. ตรวจสอบว่า Order ต่อไปอยู่ใน Queue
9. คืนค่า Order
10. End

ระดับ Priority

Express > Normal > Large

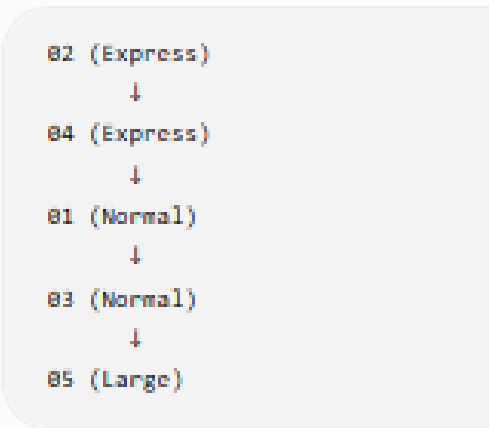
เหตุผลของการทำแบบนี้

- Express ต้องได้รับความสำคัญสูงสุด
- Normal หรือ Large สามารถรอได้
- ถ้า Priority เท่ากันสามารถใช้ Order Time ตัดสินได้
- ลดเวลารอของ Order ที่เร่งด่วน

ข้อจำกัด

- โครงสร้างซับซ้อนกว่า Algorithm A
- Normal หรือ Large อาจรอนาน
- อาจเกิดปัญหา Starvation ถ้ามี Express เข้ามาตลอด
- ต้องจัดการ Priority และ Order Time ให้ถูกต้อง

ตัวอย่างลำดับการทำงาน



ดังนั้นลำดับคือ

02 → 04 → 01 → 03 → 05